02-07、接口自动化Agent开发之用例生成流程

1、单用例执行的流程设计

设计用例前置条件执行的顺序结构、支持复杂场景的测试需求。

```
执行修改用户个人信息用例
├─ 步骤：前置步骤
│   ├─ 调用登录接口获取 token(系统A的接口)
│   │   ├─ 请求信息：包含认证所需的用户名/密码
│   │   ├─ 响应内容：提取token用于后续请求
│   ├─ 获取用户基础信息(系统B的接口)
│   │   ├─ 请求信息：携带上一步获取的token
│   │   ├─ 响应内容：提取用户ID等关键信息
├─ 步骤：执行主接口请求
│   ├─ 请求信息：组装完整请求（URL+Headers+Body）
│   ├─ 响应内容：接收原始响应数据
├─ 步骤：响应断言
│   ├─ HTTP状态码验证
│   ├─ 业务状态码验证
│   ├─ 关键字段存在性及类型验证
├─ 步骤：数据库断言
│   ├─ 执行SQL查询
│   ├─ 比对数据库实际值与预期值
```

关键设计要点：

1. **步骤解耦**：每个用例依赖独立，支持并行执行
2. **数据传递**：前置步骤的响应数据可通过变量注入后续步骤
3. **异常处理**：任一步骤失败立即终止并生成详细错误报告

2、用例的数据格式设计

```
{
    "name": "用例名称",
    "description": "用例描述",
    "project": "所属项目",
    "module": "所属模块",
    "interface": "接口名称",
    "preconditions": [
        {
            "name": "前置步骤名称",
            "request": {
                "interface_id": "接口ID",
                "method": "HTTP方法",
                "url": "接口路径",
                "base_url": "基础URL",
                "headers": {"头信息"},
                "params": {"查询参数"},
```

```
                "body": {"请求体"}
            },
            # 数据提取
            "extract": [
                ["字段名称", "接口返回的响应字段路径(jmespath提取表达式)", ]
            ]
        }
    ],
    "request": {
        "interface_id": "主接口ID",
        "name": "主请求名称",
        "method": "HTTP方法",
        "url": "接口路径",
        "base_url": "基础URL",
        "headers": {"请求头信息"},
        "params": {"查询参数"},
        "body": {"请求体"}
    },

    "assertions": {
        "response": [
            {
                "type": "断言类型(equal/not_null/http_code),http_code为响应状态码",
                "field": "响应字段路径(jmespath提取表达式)，当类型为http_code时，该字段为,HTTP状态码",
                "expected": "预期值"
            }
        ],
        "database": [
            {
                "query": "SQL查询语句",
                "expected": {"字段": "预期值"}
            }
        ]
    }
}
```

字段详解:

1. **前置条件**:
   - 支持多级依赖（如：登录→获取配置→准备数据）
   - `extract` 实现跨接口数据传递
2. **请求配置**:
   - 支持动态变量（`{{variable}}` 语法）
   - 完整HTTP协议要素支持
3. **断言机制**:
   - 响应断言：支持状态码、字段值验证
   - 数据库断言：支持多表联合查询验证

3、基本用例生成的提示词

```
# @Author  : 木森
# @weixin: python771
from langchain_core.prompts import PromptTemplate
```

```
from agents.models import deepseek_v3_llm
from rag.data_storage.sql_storage import SQLDB

prompt = PromptTemplate(
    input_variables=["api_info", 'all_api'],
    template="""
    角色：软件测试工程师和接口测试专家
    背景：用户需要根据提供的用例信息和接口文档，生成符合特定结构的测试用例。
    用户正在开发一个Web测试平台，需要对用户登录功能进行测试，同时需要确保测试用例的结构化和标准化。
    根据提供的项目接口信息和接口文档，按输出要求的格式生成测试用例，需要写出每条的用例前置条件，
    并且把前置条件中所涉及到的依赖数据相关的接口，按照调用顺序输出在每条用例数据中
    确保测试用例的完整性和准确性。

    项目所有接口如下所示：
        {all_api}
    你需要对下面这个接口进行用例编写：
    接口文档信息：
        {api_info}
    输出测试用例要求格式：
        测试用例名称：用户登录功能测试（用户名+密码）
        测试步骤：
            调用接口 /api/users/login，请求方法为 POST。
            请求体中传入正确的请求参数：
            检查响应状态码是否为 200。
            检查响应体中是否返回 token，并验证其为字符串类型。
        预期结果：
            接口返回状态码为 200；
            响应体中包含非空 token；
        前置依赖数据的接口调用顺序：
            ["接口A","接口B"]
    """
)

api_info = """
{'version': None, 'description': '创建项目', 'path': '/api/pro/projects/', 'tag': None, 'responses': '{"201": {"id": {"type": "integer", "description": "项目id"}, "name": {"type": "string", "description": "项目名称"}, "user_id": {"type": "integer", "description": "项目负责人"}, "create_time": {"type": "string", "description": "创建时间"}}, "422": {"detail": {"type": "array", "description": ""}}}', 'summary': '创建项目', 'method': 'POST', 'module': None, 'requestBody': '{"name": {"type": "string", "description": "项目名称"}, "user": {"type": "integer", "description": "项目负责人"}}', 'project': 'web测试平台', 'parameters': '[]', 'distance': -3.5762786865234375e-07}
"""

with SQLDB() as db:
    sql = 'SELECT summary,description FROM api_interface WHERE project="web测试平台"'
    res = db.cursor.execute(sql)
    all_apis = db.cursor.fetchall()
    print(all_apis)
chain = prompt | deepseek_v3_llm

response = chain.invoke({"api_info": api_info, "all_api": all_apis})
print(response.content)
```

生成策略：

1. **场景覆盖**：
   - 正常流程（Happy Path）
   - 参数异常（空值/超长/非法格式）
   - 业务异常（重复创建/权限不足）
2. **依赖分析**：
   - 自动识别鉴权依赖
   - 检测数据准备依赖（如需要先创建资源）

4、结构化用例生成提示词

```
# @Author  : 木森
# @weixin: python771
from langchain_core.prompts import PromptTemplate

from agents.models import deepseek_v3_llm
from rag.data_storage.sql_storage import SQLDB

prompt = PromptTemplate(
    input_variables=['input_format', "case_info", 'case_api', 'other_api',
'test_data'],
    template="""
    角色: 软件测试工程师和接口测试专家
    背景: 用户需要根据提供的用例信息和接口文档，生成符合特定结构的测试用例。用户正在开发一个web测试平台，需要对用户登录功能进行测试，同时需要确保测试用例的结构化和标准化。
    你是一位经验丰富的软件测试工程师，专注于接口测试领域。你熟悉各种测试框架和工具，能够根据接口文档快速生成高质量的测试用例。你对HTTP协议、JSON数据格式以及测试用例设计的最佳实践有着深刻的理解。你具备以下关键能力：
      - 熟练掌握HTTP协议和RESTful API的设计原则。
      - 精通JSON数据格式的解析和生成。
      - 熟悉测试用例设计方法，如等价类划分、边界值分析等。
      - 能够使用JMESPath进行JSON数据的提取和断言。
      - 熟悉SQL语言，能够进行数据库验证。
     根据提供的用例信息和接口文档，生成符合要求的测试用例，确保测试用例的完整性和准确性。

    # 一、输出的用例数据结构要求如下:
        {input_format}
        说明如果前置依赖接口请求后头提取数据，在后面相关接口参数的或者请求头中引用时使用，按照{{字段名称}}格式引用提取的字段值
        比如提取token字段：
           "extract": [
                ["token", "data.token"]
            ]
        在请求头中使用token时则按照下面的语法：
            "headers": {{"token":{{{{token}}}}}},

    # 二、用户提供的输入数据如下：
        1、测试用例的基本信息
            {case_info}
        2、用例对应接口的文档
            {case_api}
        3、用例前置依赖相关的接口：
            {other_api}
    # 三、补充数据：
```

```
        用户提前准备的测试数据如下：{test_data}

    """,
)

# 输出数据的格式要求
format_input = {
    "name": "用例名称",
    "description": "用例描述",
    "project": "所属项目",
    "module": "所属模块",
    "interface": "接口名称",
    "preconditions": [
        {
            "name": "前置步骤名称",
            "request": {
                "interface_id": "接口ID",
                "method": "HTTP方法",
                "url": "接口路径",
                "base_url": "基础URL",
                "headers": {"头信息"},
                "params": {"查询参数"},
                "body": {"请求体"}
            },
            # 数据提取
            "extract": [
                ["字段名称", "接口返回的响应字段路径(jmespath提取表达式)", ]
            ]
        }
    ],
    "request": {
        "interface_id": "主接口ID",
        "name": "主请求名称",
        "method": "HTTP方法",
        "url": "接口路径",
        "base_url": "基础URL",
        "headers": {"请求头信息"},
        "params": {"查询参数"},
        "body": {"请求体"}
    },

    "assertions": {
        "response": [
            {
                "type": "断言类型(equal/not_null/http_code),http_code为响应状态码",
                "field": "响应字段路径(jmespath提取表达式)，当类型为http_code时，该字段
为,HTTP状态码",
                "expected": "预期值"
            }
        ],
        "database": [
            {
                "query": "SQL查询语句",
                "expected": {"字段": "预期值"}
            }
        ]
    }
}
```

```
# 基础的测试用例数据
case_info = """
    测试用例名称：创建项目功能测试（正常场景）
    测试步骤：
        1. 调用接口 /api/pro/projects/，请求方法为 POST。
        2. 请求体中传入有效的项目名称和负责人ID：
           {"name": "TestProject", "user": 1}
        3. 检查响应状态码是否为 201。
        4. 检查响应体中是否返回项目ID、名称、负责人ID和创建时间，并验证其数据类型。
    预期结果：
        1. 接口返回状态码为 201；
        2. 响应体中包含非空的项目ID(integer)、项目名称(string)、负责人ID(integer)和创建时间(string)；
    前置依赖数据的接口调用顺序：
        ["用户登录"]
"""

# 用例接口
case_api = {'version': None,
            'description': '创建项目',
            'responses': '{"201": {"id": {"type": "integer", "description": "项目id"}, "name": {"type": "string", "description": "项目名称"}, "user_id": {"type": "integer", "description": "项目负责人"}, "create_time": {"type": "string", "description": "创建时间"}}, "422": {"detail": {"type": "array", "description": ""}}}',
            'method': 'POST', 'summary': '创建项目', 'path': '/api/pro/projects/', 'module': None,
            'requestBody': '{"name": {"type": "string", "description": "项目名称"}, "user": {"type": "integer", "description": "项目负责人"}}',
            'project': 'web测试平台', 'parameters': '[]'}

# 其他接口
other_api = [
    {'version': None, 'description': '登录的逻辑,实现用户名、手机号、邮箱均可以作为账号登录', 'path': '/api/users/login',
     'method': 'POST', 'tag': None, 'summary': '用户登录',
     'responses': '{"200": {"token": {"type": "string", "description": "访问令牌"}, "user": {"id": {"type": "integer", "description": "用户id"}, "username": {"type": "string", "description": "用户名"}, "nickname": {"type": "string", "description": "昵称"}, "email": {"type": "string", "description": "邮箱"}, "mobile": {"type": "string", "description": "手机号"}, "is_active": {"type": "boolean", "description": "是否激活"}, "is_superuser": {"type": "boolean", "description": "是否超级管理员"}}}, "422": {"detail": {"type": "array", "description": ""}}}',
     'module': None,
     'requestBody': '{"username": {"type": "string", "description": "用户名"}, "password": {"type": "string", "description": "密码"}}',
     'project': 'web测试平台', 'parameters': '[]'}
]

# 测试数据
test_data = {
    "用户登录的测试账号": {
        "username": "python771",
        "password": "musen12399"
    },
    "项目的base_url地址": {
```

```
        "base_url": "http://124.222.144.182:8888"
    }
}

if __name__ == '__main__':
    chain = prompt | deepseek_v3_llm
    response = chain.invoke({
        "input_format": format_input,
        "case_info": case_info,
        "case_api": case_api,
        "other_api": other_api,
        "test_data": test_data
    })
    print(response.content)
```

字段详解：

1. **前置条件**：
   - 支持多级依赖（如：登录→获取配置→准备数据）
   - `extract` 实现跨接口数据传递
2. **请求配置**：
   - 支持动态变量（`{{variable}}` 语法）
   - 完整HTTP协议要素支持
3. **断言机制**：
   - 响应断言：支持状态码、字段值验证
   - 数据库断言：支持多表联合查询验证